Übung 2

CSS Grundlagen

Einführung in Webengineering · WS 2026/27

Sie gestalten HTML-Dokumente mit CSS, untersuchen die Wirkung von Regeln und erklären das Verhalten von Boxen. Prüfen Sie Ihre Vermutungen anschließend im Browser.

01 Inhalt und Gestaltung trennen

a) Erläutern Sie die Aufgaben von HTML, CSS und JavaScript. Nennen Sie zwei Vorteile, die eine gemeinsame externe CSS-Datei für mehrere HTML-Seiten bietet.

b) Kennzeichnen Sie in der folgenden Regel Selektoren, Deklarationen, Eigenschaften und Werte. Welche Bedeutung haben das Komma, der Doppelpunkt und das Semikolon?

```
h1, h2 {
  color: #154a6c;
  border-bottom: 2px solid currentColor;
}
```

c) Kopieren Sie Ihre Lerngruppen-Webseite aus Übung 1 in den Ordner `kapitel02`. Legen Sie `css/styles.css` an, binden Sie die Datei in `index.html` und `material.html` ein und testen Sie die Regel aus b). Geben Sie außerdem an, wie sich die Textfarbe einer Überschrift mit einem `style`-Attribut beziehungsweise einem `style`-Element setzen lässt.

02 Selektoren und Kombinatoren anwenden

Untersuchen Sie diesen HTML-Ausschnitt. Die IDs dienen zur eindeutigen Angabe Ihrer Ergebnisse.

```
<section id="kurs">
  <h2>Themen</h2>
  <p id="p1" class="info">HTML</p>
  <div><p id="p2">CSS</p></div>
  <p id="p3" class="info extra">Übung</p>
  <a id="a1" href="material.pdf">Materialien</a>
</section>
```

a) Welche Elemente wählen die Selektoren `.info`, `section p`, `section > p`, `h2 + p`, `h2 ~ p` und `[href$=".pdf"]` jeweils aus? Geben Sie deren IDs an. Erläutern Sie anhand der Treffer die Unterschiede zwischen den vier Kombinatoren Leerzeichen, `>`, `+` und `~`.

b) Formulieren Sie je einen Selektor für das Element mit der ID `p2`, für Elemente mit beiden Klassen `info` und `extra` sowie für Links, deren Ziel mit `https://` beginnt.

c) Fügen Sie den Ausschnitt in ein vollständiges Testdokument ein. Prüfen Sie die Selektoren nacheinander mit einer sichtbaren Rahmenfarbe. Notieren Sie eine Beobachtung, bei der eine Verwechslung von Kind und Nachfahre zu einem anderen Ergebnis führt.

Zustände und konkurrierende Regeln

03 Pseudoklassen und Pseudoelemente nutzen

a) Gestalten Sie Links Ihrer Lerngruppen-Webseite mit `:hover` und `:focus`. Der Tastaturfokus soll durch eine gut sichtbare `outline` erkennbar sein. Testen Sie die Navigation mit Maus und Tabulatortaste und erläutern Sie den Unterschied der beiden Zustände.

b) Vergleichen Sie `section > p:first-child` mit `section > p:nth-child(2)` am Beispiel aus Aufgabe 2. Welche Elemente werden ausgewählt? Welche Geschwister werden mitgezählt?

c) Stellen Sie dem Text der Absätze mit der Klasse `info` mit `::before` ein dekoratives Zeichen voran. Welche Rolle hat dabei `content`? Erklären Sie den Unterschied zwischen Pseudoklasse und Pseudoelement.

d) Wählen Sie mit `:has()` Abschnitte aus, die einen Link auf eine PDF-Datei enthalten. Recherchieren Sie die Browserunterstützung und testen Sie den Selektor. Notieren Sie Browser und Version.

04 Kaskade und Spezifität erklären

Für dieses Beispiel gelten ausschließlich die folgenden Autorenregeln, ohne Kaskadenebenen, Animationen oder Übergänge. Die CSS-Regeln stehen in der gezeigten Reihenfolge.

```
<main id="kurs"><p class="info">Nächster Termin</p></main>
```

```
main          { color: black; }
p             { color: gray; }
.info         { color: green; }
#kurs .info   { color: blue; }
p.info        { color: red; }
```

a) Bestimmen Sie die Textfarbe des Absatzes und die Spezifität aller fünf Selektoren als Tupel (IDs, Klassen/Attribute/Pseudoklassen, Typen/Pseudoelemente). Warum entscheidet hier nicht allein die letzte Regel?

b) Ergänzen Sie am Absatz `style="color: purple"`. Ergänzen Sie danach zusätzlich `!important` bei `color: green`. Bestimmen und begründen Sie die Farbe nach jedem Schritt.

c) Testen Sie unabhängig von a) und b) zwei gleich spezifische Regeln für `p`: eine externe CSS-Datei setzt Blau, ein `style`-Element Rot. Welche Farbe gewinnt, wenn `link` vor beziehungsweise nach `style` steht? Entfernen Sie danach alle Farbangaben am Absatz; lassen Sie nur `main { color: black; }` stehen. Welche Farbe wird vererbt?

05 Farben und Transparenz untersuchen

a) Vergleichen Sie `#3399cc`, `#39c` und `rgb(51, 153, 204)`. Beschreiben sie dieselbe Farbe? Erläutern Sie zusätzlich die drei Bestandteile von `hsl(200, 60%, 50%)`.

b) Testen Sie eine Box mit `background-color: rgba(51, 153, 204, 0.5);` auf hellem und dunklem Hintergrund. Erklären Sie den Alpha-Wert und die unterschiedliche sichtbare Farbe. Ersetzen Sie anschließend die Hintergrundfarbe der Box durch `background-color: transparent;` und beschreiben Sie die Veränderung.

c) Wählen Sie eine gut lesbare Kombination aus Text- und Hintergrundfarbe für Ihre Webseite. Prüfen Sie den Kontrast mit den Entwicklerwerkzeugen und halten Sie Farben sowie das angezeigte Kontrastverhältnis fest.

Größen und Elementfluss verstehen

06 Mit relativen Maßeinheiten rechnen

Es gelten `html { font-size: 16px; }` und ein Elterncontainer mit `font-size: 20px` sowie `600px` Inhaltsbreite. Die folgende Blockbox ist sein direktes Kind. Viewport: `1200px` breit, `800px` hoch.

```
.karte {
  font-size: 1.5em; width: 50%;
  padding: 1em; margin-left: 2rem;
}
```

a) Berechnen Sie Schriftgröße, Inhaltsbreite, Innenabstand und linken Außenabstand in CSS-Pixeln. Warum hat `em` bei `font-size` und `padding` unterschiedliche Bezugsgrößen? Es gilt `content-box`.

b) Berechnen Sie `10vw`, `25vh` und `5vmin`. Was ändert sich, wenn die Viewportbreite auf `800px` sinkt, seine Höhe aber gleich bleibt?

c) Ändern Sie nur die Wurzelschriftgröße auf `20px` und berechnen Sie die Werte aus a) erneut. Wann ist `rem` für eine gemeinsame Abstands- und Schriftgrößenskala hilfreich?

07 Boxmodell und Abstände prüfen

Für eine Blockbox gelten die folgenden Angaben. Ihr Elterncontainer ist ausreichend breit.

```
.box {
  width: 300px; padding: 20px;
  border: 5px solid #154a6c; margin: 10px;
  box-sizing: content-box;
}
```

a) Skizzieren Sie die vier Bereiche der Box. Berechnen Sie die Inhaltsbreite, die Breite einschließlich Rahmen und den Platzbedarf einschließlich beider horizontaler Außenabstände.

b) Wechseln Sie zu `border-box` und berechnen Sie die drei Größen erneut. Kontrollieren Sie beide Varianten in den Entwicklerwerkzeugen. Ändert `outline: 3px solid red` die berechneten Größen?

c) Zwei direkt benachbarte Blockabsätze im normalen Blockfluss haben `margin-bottom: 24px` beziehungsweise `margin-top: 16px`. Wie groß ist ihr Abstand bei kollabierenden Margins? Prüfen und erklären Sie das Ergebnis.

08 Elementfluss und Textumfluss vergleichen

a) Legen Sie drei benachbarte `span`-Elemente mit kurzen Texten an. Geben Sie ihnen `width: 120px`, `height: 60px` und einen Rahmen. Vergleichen Sie `display: inline`, `block` und `inline-block`: Welche Maße wirken, und wie werden die Boxen angeordnet? Verkleinern Sie auch das Browserfenster.

b) Setzen Sie für das mittlere Element `display: none`. Was geschieht mit seinem Platz im Layout? Bleibt das Element im DOM vorhanden?

c) Setzen Sie für ein `120px` breites Bild `float: left` und lassen Sie Text darum fließen. Setzen Sie die nächste Überschrift mit `clear: both` unter das Bild. Worin liegt der Unterschied zu `inline-block`?

Positionierung und eigene Gestaltung

09 Positionen im Browser nachvollziehen

Erstellen Sie eine Testseite mit drei beschrifteten Blockboxen. Die mittlere Box erhält die Klasse `markiert`. Ein gemeinsamer Container mit der Klasse `rahmen` umfasst alle drei Boxen.

```
.rahmen   { position: relative; border: 2px solid gray; }
.markiert { position: relative; top: 20px; left: 30px; }
```

a) Vergleichen Sie `position: static` und `position: relative` bei der mittleren Box. Welche Wirkung haben `top` und `left`? Was passiert jeweils mit dem ursprünglichen Platz der Box und der Position der nachfolgenden Box?

b) Wechseln Sie bei der mittleren Box zu `position: absolute; top: 0; right: 0; left: auto;`. Begrenzen Sie ihre Breite auf `8rem`. Welcher Bereich dient nun als Bezug für die Positionierung? Wie verändert sich der normale Elementfluss?

c) Erzeugen Sie genügend Inhalt zum Scrollen. Positionieren Sie einen Link mit `position: fixed; bottom: 1rem; right: 1rem;`. Beobachten Sie ihn beim Scrollen und erklären Sie den Unterschied zur absoluten Positionierung im Container.

d) Testen Sie unabhängig davon eine Navigationsleiste mit `position: sticky; top: 0;` am Anfang eines langen Containers. Scrollen Sie über dessen unteres Ende hinaus. Beschreiben Sie, wann die Leiste haftet und wodurch ihr Bewegungsbereich begrenzt wird.

10 Die Lerngruppen-Webseite gestalten

Führen Sie die Erkenntnisse auf Ihrer Webseite aus Aufgabe 1 zusammen. Gestalten Sie `index.html` und `material.html` über die gemeinsame Datei `css/styles.css`. Verwenden Sie die behandelten Grundlagen; Flexbox, Grid und JavaScript sind für diese Aufgabe nicht erforderlich.

a) Gestalten Sie beide Seiten einheitlich: gut lesbare Schrift mit einer Ersatzschrift, abgestufte Überschriften, gut unterscheidbare Links und passende Abstände. Begrenzen Sie die Inhaltsbreite auf `60rem` und zentrieren Sie den Inhalt. Bei schmalem Fenster sollen links und rechts Abstände bleiben.

b) Stellen Sie die Navigationslinks als `inline-block` mit Innenabstand dar. Sie sollen bei Platzmangel umbrechen. Gestalten Sie Mauszeiger- und Tastaturfokuszustände und sorgen Sie dafür, dass das Bild in seinem Container bleibt und sein Seitenverhältnis behält.

c) Geben Sie dem Kontaktabschnitt eine eigene Gestaltung über eine Klasse. Ergänzen Sie darin einen kurzen sichtbaren Hinweis, etwa „Nächstes Treffen“, und positionieren Sie ihn rechts oben im Abschnitt. Planen Sie genügend Innenabstand ein, damit der Hinweis keinen Text verdeckt.

d) Testen Sie beide Seiten bei ungefähr `375px` und `1200px` Viewportbreite sowie bei `200%` Browserzoom. Prüfen Sie Umbrüche, Überlappungen, horizontales Scrollen und die Tastaturbedienung. Korrigieren Sie Probleme und dokumentieren Sie drei konkrete Gestaltungsentscheidungen mit den zugehörigen CSS-Regeln.

Zum Abschluss: Halten Sie Ihre Antworten und Beobachtungen fest. Die beiden gestalteten Seiten, die CSS-Datei und die Testbeispiele sollen als bearbeitbare Dateien vorliegen.